Claude Code - Complete Course Notes

Table of Contents

- What is Claude Code?
- Setup & Installation
- Terminals & Platform Differences
- Base Usage & IDE Integration
- Desktop App & Mobile App
- Configuring Claude Code
- Choosing AI Models
- Sessions & Context Window
- When to Start New Sessions & Compaction
- Core Features You May Not Know
- Advanced Permissions Management
- Docker Sandbox
- Native Sandboxing
- Undoing Actions & Version Control
- Commands, Shortcuts & Settings Cheat Sheet
- Prompt & Context Engineering
- Working with Specs
- Prompt Engineering Best Practices
- Initializing Claude Projects (`/init`)
- CLAUDE.md Files
- CLAUDE.md vs Auto Memory
- Plan Mode
- Built-in Tools & MCP Servers
- Subagents
- Creating Custom Subagents
- Encouraging Agent Usage
- Agent Skills
- Adding Custom Skills
- Skills as Commands
- Enhancing Skills & Third-Party Skills
- Using Screenshots for Feedback
- Hooks
- Plugins
- Browser Access (Playwright)
- Automated Tests as Feedback Loops
- The RALF Loop (Autonomous Mode)
- Claude Code Web (Cloud)

What is Claude Code?

Claude Code is a **CLI (Command Line Interface) tool** by Anthropic that generates, edits, and manages code using AI. It operates directly on your project files and can execute commands, run tests, and interact with your codebase.

Two main ways to use it:

- **Vibe coding** - let the AI do everything with minimal oversight
- **Agentic engineering** - you stay in control, give clear instructions, review code, and steer direction (recommended)

> **vs GitHub Copilot:** Copilot is primarily an inline autocomplete tool integrated into your editor. Claude Code is a CLI-first agent that can autonomously read, write, and execute across your entire project. Copilot Chat is the closest equivalent, but Claude Code operates at a deeper project level.

Setup & Installation

- Available for **all operating systems** (macOS, Windows, Linux)
- Primarily a command line tool, though it integrates with code editors
- Run `claude` in your terminal to start a session
- First time: it will ask whether you trust the folder (Claude Code reads files, writes code, and executes commands in that project)

Terminals & Platform Differences

Claude Code runs in any terminal, but **keyboard shortcuts differ** by platform and terminal:

Action	Shortcut (varies)
New line	`SHIFT + ENTER` or `OPTION + ENTER` or `ALT + ENTER`
Switch modes	`SHIFT + TAB`
Cancel	`CTRL + C`

Always consult the official docs for your specific terminal/platform.

Base Usage & IDE Integration

VS Code Integration

- Install the **official Claude Code extension** by Anthropic from the VS Code marketplace
- Gives you a **diff viewer** showing proposed changes (left = original, right = proposed)
- Accept or reject changes visually
- Open the **command palette** and type "Claude Code" to access a GUI panel for conversations
- Can attach files, paste images, browse past conversations

> **vs GitHub Copilot:** Copilot integrates via its own VS Code extension with inline suggestions and Copilot Chat panel. Claude Code's VS Code integration similarly provides a chat panel, but Claude Code also works fully from the terminal without any IDE.

Permission Modes (when editing files)

- **Yes** - allow this single edit
- **Yes, allow all edits during session** - stop asking for this session
- **Escape** - reject the edit and provide follow-up instructions

---

Desktop App & Mobile App

- **Desktop app** available for macOS and Windows — combines Claude Chat + Claude Code
- **Mobile app** — manage and monitor cloud tasks (dispatch/monitor, not for local coding)
- Everything you learn about prompts, skills, and context applies across CLI, desktop, and mobile

---

Configuring Claude Code

Three levels of configuration (in priority order):

Level	Location	Purpose
Global	`~/.claude/settings.json`	System-wide settings
Project	`.claude/settings.json`	Project-specific, checked into git
Local	`.claude/settings.local.json`	Project-specific, NOT checked into git (personal overrides)

Important settings:

- **Permissions** — deny reading `.env` files to protect secrets:

```
{
  "permissions": {
    "deny": {
      "read": ["**/.env"],
      "write": ["**/.env"],
      "bash": ["**/.env"]
    }
  }
}
```

- `alwaysThinkingEnabled` — enables advanced thinking mode (on by default; turning off yields worse results)
- Use `/config` inside Claude Code to browse and change settings interactively

> **vs GitHub Copilot:** Copilot uses VS Code settings and `.github/copilot-instructions.md` for configuration. Claude Code has a more granular, multi-level settings system with explicit permission controls.

Choosing AI Models

- Use `/model` command or `OPTION + P` (macOS) / `ALT + P` (Windows/Linux) to switch models
- Available models: **Opus** (most capable), **Sonnet** (balanced), **Haiku** (fastest/cheapest)
- Can also set via `--model opus` flag or in `settings.json` as `"model": "opus"`

Sessions & Context Window

Context window = the amount of information (measured in tokens) the AI can "remember" in a single conversation.

- Claude Code Opus has ~200,000 tokens available
- Some space is pre-occupied by: system prompt, tool descriptions, MCP tools
- An **auto-compact buffer** is reserved so Claude Code can automatically summarize when the window fills up

Key commands:

Command	Purpose
`/clear`	Clear context and start fresh
`/context`	View context window usage stats
`/usage`	Check remaining plan usage
`/compact`	Manually trigger compaction (summarization)

- You can run **multiple Claude Code sessions in parallel** on the same project (just avoid them editing the same files)

When to Start New Sessions & Compaction

Compaction = when context fills up, Claude Code generates a summary of the conversation and discards old context. Some details will be lost.

Start a new session when:

- Working on a new, unrelated feature
- The AI gets stuck and can't fix a problem
- You want a fresh context window for a complex task

You can trigger compaction manually with `/compact` before a big task to free up space.

Core Features You May Not Know

Inline prompt at startup

```
claude "explain this project"
```

Starts Claude Code and immediately processes that prompt.

Non-interactive mode (`-p` flag)

```
claude -p "explain this project"
```

Runs in the background, outputs the answer, then exits. No interactive session.

Resume sessions

- `/resume` inside Claude Code — browse and resume any past session
- `claude -c` — continue the most recent session

> **vs GitHub Copilot:** Copilot Chat doesn't persist sessions across restarts. Claude Code maintains full session history that you can resume anytime.

Advanced Permissions Management

Three permission modes (cycle with `SHIFT + TAB`):

Mode	What it allows
Default	Asks permission for everything
Accept Edits	Auto-accepts file edits in your project, still asks for bash commands, git operations, etc.
Plan Mode	Read-only; creates a plan without editing files

Skip all permissions:

```
claude --dangerously-skip-permissions
```

Warning: Grants all permissions. The AI could theoretically write and execute a destructive script. Use with sandbox mode.

Docker Sandbox

Run Claude Code in an **isolated Docker container** so it can't affect your system:

```
docker sandbox claude
```

- Automatically starts in dangerous-skip-permissions mode (safe because it's sandboxed)
- The AI can't access anything outside the project
- Ideal for hands-off, autonomous work

Native Sandboxing

Claude Code has a **built-in sandbox mode** (no Docker needed):

- Run `/sandbox` inside Claude Code to enable it
- Scopes access to the project folder only
- Controls network access to prevent data exfiltration
- Adds a `"sandbox"` entry to your settings file
- All future sessions run in sandbox mode

Undoing Actions & Version Control

Two ways to undo:

1. **`ESC + ESC`** — press Escape twice to rewind to a previous point in the conversation
2. **`/rewind`** — browse session history and rewind

Version control (Git) is essential:

- Commit frequently when working with AI
- Use git's diff tools to review AI changes
- AI can mess up or delete files — git lets you recover
- The rewind feature can be buggy; git is your safety net

Commands, Shortcuts & Settings Cheat Sheet

CLI Commands & Flags

Command	Description
`claude` / `claude "prompt"`	Start a new session, optionally with a prompt
`claude -p "prompt"`	Run prompt and quit (no interactive session)
`claude -c`	Continue most recent session
`claude --agent DocsExplorer`	Start with a custom agent
`claude --allowedTools "Read" "Write"`	Skip permission dialogs for specified tools
`claude --disallowedTools "Write"`	Remove tools from Claude Code's context
`claude --dangerously-skip-permissions`	Skip ALL permission dialogs
`claude --append-system-prompt "..."`	Add instruction to the system prompt

Command	Description
`claude --model opus`	Set model for this session
`claude --permission-mode plan`	Start in plan mode
`claude --remote "prompt"`	Start a remote (cloud) session
`claude --system-prompt "..."`	Replace the entire system prompt

Keyboard Shortcuts

Shortcut	Action
`SHIFT + ENTER` / `OPTION + ENTER` / `CTRL + J`	New line
`SHIFT + TAB`	Cycle modes (default → accept edits → plan)
`CTRL + C`	Cancel input or generation
`ESC`	Cancel generation (inject a new prompt mid-task)
`ESC + ESC`	Rewind/undo last action
`OPTION + P` / `ALT + P`	Switch model
`Arrow keys`	Cycle options or past messages
`CTRL + O`	Toggle verbose output
`CTRL + B`	Move task to background
`CTRL + V` / `CMD + V`	Paste text or image

Slash Commands

Command	Purpose
`/help`	List commands and usage help
`/model`	Choose active model
`/clear`	Clear session context
`/compact`	Summarize and clear context
`/config`	Interactive settings menu
`/context`	View context window stats
`/usage`	View plan usage
`/init`	Analyze project and create CLAUDE.md
`/mcp`	Manage MCP servers
`/permissions`	View/update permissions

Command	Purpose
`/rewind`	Undo to earlier point
`/statusline`	Configure status line
`/teleport`	Resume a remote session

Prompt & Context Engineering

Good prompts = **specific instructions** + **relevant context** (without unnecessary noise).

Key principles:

- Better input → better output
- Include relevant context (files, docs) but avoid irrelevant information
- Think and plan before prompting

Working with Specs

Creating a specification document:

1. Describe your application to an AI (e.g., ChatGPT) to generate a technical spec
2. Save it as `spec.md` in your project root
3. Reference it in prompts with `@spec.md`

The `@` syntax:

Use `@filename` to explicitly include a file in your prompt's context. Claude Code can find files on its own, but explicitly pointing at known-relevant files improves results.

Providing external context:

- Copy documentation articles into your prompt
- Use XML tags to wrap longer pieces of text for clarity:

```
<betterauth-docs>
...pasted documentation...
</betterauth-docs>
```

Prompt Engineering Best Practices

Practice	Description
Concise & Precise	Describe the task clearly without fluff
No Unnecessary Context	Only reference files you KNOW matter, not files you THINK might matter

Practice	Description
Think, Plan, Prompt	Plan before typing; don't "fix stuff over time"
Don't Test the AI	If you know about a pitfall, tell the AI upfront — don't hide information
Explicitly Name Tools	If you know a specific tool/MCP/skill should be used, say so in the prompt

Main theme: YOU are in control. YOU steer the AI.

Initializing Claude Projects (`/init`)

Run `/init` in a new project to have Claude Code:

- Thoroughly analyze the codebase (dependencies, file structure, config files)
- Detect relevant files like `spec.md`
- Generate a `CLAUDE.md` file with project-specific instructions

> **vs GitHub Copilot:** Copilot uses `.github/copilot-instructions.md` for project context. Claude Code uses `CLAUDE.md` with richer, hierarchical support (nested files in subfolders).

CLAUDE.md Files

A file placed in your project root that is **automatically loaded into every new Claude Code session**.

What to put in it:

- Runtime/tooling info (e.g., "We use Bun, not Node.js")
- Architecture overview (keep it brief)
- References to other docs (e.g., "See @spec.md for detailed architecture")
- Instructions for AI behavior (e.g., "Keep replies short, no long code snippets")

Key principles:

- Keep it **concise** — everything in it consumes context tokens every session
- Point to detailed docs with `@filename` rather than inlining everything
- Add guidance on **when** to consult referenced files

Nested CLAUDE.md files:

- Place in subfolders (e.g., `app/CLAUDE.md`)
- Only loaded when Claude Code works on files in that subfolder
- Can have as many levels of nesting as needed

> **vs GitHub Copilot:** `copilot-instructions.md` serves a similar purpose — project-level context instructions. But Claude Code supports **nested** CLAUDE.md files per folder, giving more granular control. Copilot has a single flat instructions file.

CLAUDE.md vs Auto Memory

Feature	CLAUDE.md	Auto Memory
Maintained by	You (manual)	Claude Code (automatic)
Stored in	Project root (`.claude/`)	`~/.claude/projects/<project>/memory/`
Loaded when	Every new session	First 200 lines of MEMORY.md per conversation
Purpose	Core instructions and project info	Learned preferences from your corrections
Available since	Always	v2.1.59+

Auto Memory learns from:

- Errors you call out multiple times
- Code style corrections you make
- Patterns and preferences you demonstrate

Auto Memory does NOT replace CLAUDE.md — if you know something should always be followed, put it in CLAUDE.md explicitly.

Manage auto memory with `/memory` command.

Plan Mode

A read-only mode where Claude Code **explores and plans before writing any code**.

How to use:

1. Write your prompt
2. Press `SHIFT + TAB` to switch to Plan mode
3. Submit — Claude Code will research, explore, and create a plan
4. Review the plan; provide feedback or clarification
5. Accept the plan to start implementation

Plan acceptance options:

- Clear context + auto-accept edits (recommended for most cases)
- Continue without clearing context
- Provide more information before accepting

Why use Plan Mode:

- Turns average prompts into good prompts
- Catches misunderstandings before code is written
- Allows you to add missing context or constraints
- Plans are saved locally and survive context compaction

Recommendation: Use Plan mode for everything except trivially simple tasks.

> **vs GitHub Copilot:** Copilot doesn't have a comparable planning mode. It generates code directly. Claude Code's Plan mode is unique in letting you review and refine the approach before any code is written.

Built-in Tools & MCP Servers

Claude Code has built-in tools for:

- **File operations** (read, write, edit)
- **Bash commands** (run terminal commands)
- **Web search** (find documentation online)
- **Web requests** (visit websites)

MCP (Model Context Protocol)

A standard for exposing tools and resources to AI tools. You can add MCP servers to extend Claude Code's capabilities.

Example: Context7 MCP Server — gives Claude Code easy access to official library documentation.

```
# Install globally
claude mcp add context7 --scope user
```

Using MCP in prompts:

```
Use web search or the Context7 MCP to find relevant documentation.
```

Managing MCP:

- `/mcp` — view and manage installed MCP servers
- MCP tools require permission on first use

> **vs GitHub Copilot:** Copilot has `@workspace` for codebase context and recently added MCP support. Claude Code's MCP integration is more mature, with a broader ecosystem of servers and a plugin marketplace.

Subagents

Subagents = separate AI instances that Claude Code spins up to handle specific tasks in parallel.

Key properties:

- Have their **own context window** (don't pollute the main one)
- Run in **parallel** with the main agent and other subagents
- Return a **summary** to the main agent when done

- Built-in example: the **Explore agent** (optimized for reading/understanding files)

Why subagents matter:

- Split complex work into smaller, focused tasks
- Keep the main context window clean
- Enable parallel processing (faster results)
- Dedicated "expert" for specific tasks

Creating Custom Subagents

File location:

- **Project-level:** `.claude/agents/AgentName.md`
- **Global:** `~/.claude/agents/AgentName.md`

Agent file structure:

```
---
name: DocsExplorer
description: Looks up documentation for libraries, frameworks, and tools
tools: WebSearch, WebFetch, mcp-search
model: sonnet
---

You are a documentation lookup agent...
- Execute all lookups in parallel
- Use Context7 MCP as primary source
- Fall back to web search if needed
- Look for llms.txt or Markdown docs first
```

Key fields:

Field	Purpose
`name`	Must match the filename
`description`	Helps Claude Code decide when to use this agent
`tools`	Which tools this agent can access
`model`	Which AI model to use (sonnet = cheaper, opus = smarter)

Encouraging Agent Usage

Add instructions to `CLAUDE.md` to ensure subagents get used:

```
Whenever working with any third-party library, you must look up the
official
documentation to ensure you're working with up-to-date information.
Use the DocsExplorer subagent for efficient documentation lookup.
```

This removes the guesswork — Claude Code **will** use the subagent instead of doing manual lookups in the main context.

Agent Skills

Skills = dynamically loaded pieces of context (best practices, conventions, patterns) that Claude Code discovers and uses as needed.

How skills differ from CLAUDE.md:

- CLAUDE.md is **always loaded** into every session
- Skills are **discovered and loaded dynamically** only when relevant
- Only the skill **name + description** is loaded by default; full content loads on demand

An open standard — also supported by Cursor, OpenCode, and other AI coding tools.

> **vs GitHub Copilot:** Copilot doesn't have an equivalent to skills. The closest concept is custom instructions in `copilot-instructions.md`, but that's always loaded (like CLAUDE.md), not dynamically discovered.

Adding Custom Skills

File location:

- **Project-level:** `.claude/skills/<skill-name>/SKILL.md`
- **Global:** `~/.claude/skills/<skill-name>/SKILL.md`

Skill file structure:

```
---
name: modern-best-practice-react-components
description: Build clean, modern React components that apply common best
practices and avoid common pitfalls
---

## React Component Guidelines
- Use functional components with hooks
- ...
```

Dynamic content loading with `references/`:

```
.claude/skills/modern-best-practice-react-components/
  SKILL.md
  references/
    YouDontNeedUseEffect.md
```

In `SKILL.md`, reference additional docs:

```
If working with useEffect, consult @references/YouDontNeedUseEffect.md
```

This creates **multi-level dynamic discovery**: Skill → loaded when relevant → references inside skill → loaded when specifically needed.

Optional metadata fields:

Field	Purpose
`allowed-tools`	Restrict which tools the skill can use
`model`	Which AI model to use
`context`	Set to `fork` to use a separate context window
`disable-model-invocation`	`true` = only usable as a command, not auto-discovered
`user-invocable`	`false` = only auto-discovered, not usable as a command

Skills as Commands

Skills automatically appear as **slash commands** (e.g., `/modern-best-practice-react-components`).

- **Primary use case:** Auto-discovery by Claude Code during tasks
- **Secondary use case:** Manual invocation as slash commands
- Knowledge-only skills won't do much when invoked manually (no action instructions)
- Skills with scripts or specific instructions can be useful as manual commands

---

Enhancing Skills & Third-Party Skills

If skills aren't being used:

- Tweak the **description** to be more specific about when the skill applies
- Add trigger hints: "Use this skill to optimize React components"
- Add `disable-model-invocation: true` to prevent auto-discovery (command-only)
- Add `user-invocable: false` to prevent manual invocation (auto-only)

Custom command-style skill example:

```
---
name: code-review
description: Review code for bugs, security, or performance issues
allowed-tools: Read
---

MODE: $ARGUMENTS

If MODE == BUGS: Focus ONLY on logical bugs.
If MODE == SECURITY: Focus ONLY on security issues.
...
```

Use `$ARGUMENTS` placeholder for dynamic input when invoked as a command.

Third-party skills:

- **skills.sh** — a public, free skills repository
- Install with: `npx skills add <owner/repo>`
- Always review and customize installed skills

Using Screenshots for Feedback

Claude Code has **image vision** — paste screenshots directly into prompts.

When to use screenshots:

- UI layout problems (positioning, styling issues)
- Visual bugs that are hard to describe in text
- Showing the current state of the application

When NOT to use screenshots:

- Error messages — copy/paste raw text instead (more accurate)

How to paste:

- `CTRL + V` / `CMD + V` / `ALT + V` (varies by platform)
- Images appear as attachments in your prompt

Hooks

Hooks = shell commands that execute automatically in response to Claude Code events (e.g., file edited, tool used).

Configuration location:

- In `settings.json` (global, project, or local)

Hook structure:

```
{
  "hooks": {
    "post-tool-use": [
      {
        "matcher": "edit|write",
        "hooks": [
          {
            "type": "command",
            "command": "cd \"$CLAUDE_PROJECT_DIR\" && bun run format
2>/dev/null || true"
          }
        ]
      }
    ]
  }
}
```

Available hook events:

Event	When it fires
`pre-tool-use`	Before Claude Code uses a tool
`post-tool-use`	After Claude Code uses a tool

Hook types:

Type	Purpose
`command`	Execute a bash command
`prompt`	Inject a message into the conversation
`agent`	Trigger an agent

Key variables:

- `$CLAUDE_PROJECT_DIR` — resolves to the current project directory

Practical example: Auto-format after every file edit

- Install a formatter (e.g., OxFormat)
- Add a `post-tool-use` hook matching `edit|write`
- Run the format command automatically

Use `/hooks` inside Claude Code to view and manage hooks interactively.

Plugins

Plugins = bundles of skills, commands, MCP servers, etc. that can be installed from a marketplace.

Managing plugins:

- `/plugin` — open the plugin marketplace
- Browse, install, and manage plugins
- Install scope: **global** (user), **project**, or **local**

Notable plugins:

Plugin	Purpose
Context7 MCP	Easy access to library documentation
TypeScript LSP	Better TypeScript error detection
Playwright	Browser access for testing web apps

Plugin marketplaces:

- Official Anthropic marketplace (default)
- Can add custom/company-internal marketplaces
- All official plugins are free

Browser Access (Playwright)

Install the **Playwright plugin** to give Claude Code browser access for testing web applications.

What it can do:

- Navigate to URLs
- Fill in forms
- Click buttons
- Take screenshots and analyze them
- Open multiple tabs
- Inspect network requests

Usage:

```
Test the application using the Playwright MCP.
Test all main features step-by-step.
The dev server is running on port 3000.
```

Important notes:

- **Very token-intensive** — screenshots, tool calls, and snapshots consume lots of tokens
- Creates a powerful **feedback loop**: test → find issues → fix → test again
- First-time use requires granting many permissions one by one

> **vs GitHub Copilot:** Copilot has no built-in browser testing capability. This is unique to Claude Code's extensible tool system.

Automated Tests as Feedback Loops

Give Claude Code a way to verify its work through automated testing.

Workflow:

1. Install a testing library (e.g., Vitest, Jest)
2. Ask Claude Code to set up tests for key features
3. Claude Code runs tests, detects failures, and fixes its own code
4. Creates a self-correcting feedback loop

Tips:

- Consider writing tests **before** implementation (test-first approach)
- AI may write tests that just match existing code — **review the tests**
- Combine with browser access for comprehensive validation
- You can specify exactly which tests to add

The RALF Loop (Autonomous Mode)

Named after **Ralf Wiggum** (The Simpsons) for its "naive persistence." An autonomous loop where Claude Code works through a task list without human intervention.

How it works:

1. Create a **task list** (e.g., `prd.json`) with tasks and a `passes: false` flag
2. Create a **shell script** that loops through tasks, invoking Claude Code with `-p` flag
3. Claude Code picks a task, implements it, verifies with tests/browser, marks as done
4. Loop continues until all tasks are done or max iterations reached

Requirements:

- **Sandbox mode enabled** (mandatory — Claude gets broad permissions)
- Good CLAUDE.md, skills, and agents already configured
- Playwright MCP for browser testing
- Automated tests for verification

When to use:

- Prototyping and proof-of-concepts
- Internal tools and utility software
- When you can work on something else in parallel

Caveats:

- **Very token-intensive** — can burn through your subscription quickly

- Less control — you only see results when you check in
- AI can get stuck and need manual help
- Working code is not production-ready code — **always review**
- Don't fall into the "vibe coding trap" — review for security, performance, and correctness

Monitoring:

```
# In a separate terminal, check what it's doing
claude -c
```

Claude Code Web (Cloud)

Offload tasks from your local machine to the cloud. Claude Code works on your GitHub repository remotely.

Requirements:

- Pro, Max, Teams, or Enterprise subscription
- Code in a GitHub repository
- Connect your Claude account to GitHub at `claude.ai/code`

Setup:

1. Connect GitHub repositories
2. Create a cloud environment (choose network access level: Trusted or Full)
3. Submit tasks via web interface or local CLI

From local CLI:

```
# Send a task to the cloud
claude --remote "change the main color from blue to purple"

# Continue a remote session
claude --remote -c
```

Using `&` prefix:

Add `&` before a message inside Claude Code to run it in the cloud:

```
& implement the plan
```

What happens in the cloud:

- Checks out your repository and branch
- Works on the code (can run tests)

- Creates a **new branch** and opens a **pull request** (never pushes to main directly)

Combining with Plan Mode:

1. Start locally in Plan mode
2. Review and finalize the plan
3. Hand off implementation to the cloud with `&`

Current limitations:

- Still in beta/research preview
- Can be buggy when used from local CLI
- Web interface tends to be more reliable

> **vs GitHub Copilot:** Copilot doesn't have remote/cloud task execution. GitHub Copilot Workspace is the closest concept — it can plan and implement across files but runs within the GitHub web interface, not as an offloaded background task.